

I'm doing a lab on a Python Pygame marble tilt maze. The marble accelerates toward the mouse, has friction and a speed cap, and bounces off walls.

Bug: near a wall's corner, the round marble sometimes bounces off empty space before it visually touches the wall.

I narrowed it down. I think the cause is that collision uses colliderect on the marble's square bounding box, while the marble is drawn as a circle. Relevant code only:

```
# game/game_engine.py (inside class GameEngine)
def _resolve_wall_collisions(self):
    for wall in self.walls:
        marble_rect = self.marble.rect()
        wall_rect = wall.rect()
        if marble_rect.colliderect(wall_rect):
            overlap_x = min(marble_rect.right, wall_rect.right) - max(marble_rect.left, wall_rect.left)
            overlap_y = min(marble_rect.bottom, wall_rect.bottom) - max(marble_rect.top, wall_rect.top)

            if overlap_x < overlap_y:
                if self.marble.x < wall_rect.centerx:
                    self.marble.x -= overlap_x
                else:
                    self.marble.x += overlap_x
                self.marble.vx *= -0.3
            else:
                if self.marble.y < wall_rect.centery:
                    self.marble.y -= overlap_y
                else:
                    self.marble.y += overlap_y
                self.marble.vy *= -0.3

# game/marble.py
class Marble:
    def __init__(self, x, y, radius=12):
        self.x = x
        self.y = y
        self.radius = radius
        self.vx = 0
        self.vy = 0

    def rect(self):
        return pygame.Rect(self.x - self.radius, self.y - self.radius, self.radius * 2, self.radius * 2)

# game/wall.py
class Wall:
    def __init__(self, x, y, width, height):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
```

```
def rect(self):
return pygame.Rect(self.x, self.y, self.width, self.height)
```

Task: fix ONLY this bug. Explain the root cause in 2-3 sentences. Replace the collision logic with true circle-vs-rectangle collision: find the closest point on the wall rect to the marble's center, collide only if the distance is less than the radius, push the marble out along the collision normal, and reflect only the velocity component along that normal (keep the existing 0.3 bounce damping). Handle the edge case where the center is exactly inside the rect (distance 0). Give me only the changed function(s), labeled with the file name. Don't change anything unrelated.

Wiring up menu buttons and round-reset logic.

main.py needs no changes. Exit posts a pygame.QUIT event, which your existing loop already handles. The flow is now game over screen → any key or click → menu → choose a difficulty or exit.

game.py

Add this near the top of the file, after the imports and colour constants:

```
python
```

```
# One place to tune every difficulty. Medium matches the original values.
```

```
DIFFICULTIES = {
```

```
    "Easy": {"tilt_strength": 0.9, "friction": 0.05, "time_limit_ms": 70000},
```

```
    "Medium": {"tilt_strength": 0.6, "friction": 0.02, "time_limit_ms": 45000},
```

```
    "Hard": {"tilt_strength": 0.35, "friction": 0.008, "time_limit_ms": 30000},
```

```
}
```

```
python
```

```
def __init__(self, width, height):
```

```
    self.width = width
```

```
    self.height = height
```

```
    self.max_speed = 9
```

```
    self.walls = self._build_maze()
```

```
    self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22
```

```
    self.font = pygame.font.SysFont("Arial", 26)
```

```
    self.big_font = pygame.font.SysFont("Arial", 64, bold=True) # Game Over / menu title
```

```
self._start_round("Medium")
```

python

```
def _start_round(self, difficulty):
```

```
    """Apply a difficulty and reset everything that belongs to a single round."""
```

```
    settings = DIFFICULTIES[difficulty]
```

```
    self.difficulty = difficulty
```

```
    self.tilt_strength = settings["tilt_strength"]
```

```
    self.friction = settings["friction"]
```

```
    self.time_limit_ms = settings["time_limit_ms"]
```

```
self.marble = Marble(50, 50) # resets position and velocity
```

```
self.start_ticks = pygame.time.get_ticks()
```

```
self.game_over = False
```

```
self.result = None # "solved" or "timeout"
```

```
self.finish_time_ms = None
```

```
self.in_menu = False
```

python

```
def _menu_buttons(self):
```

```
    """Return [(action, label, rect), ...]. action is a difficulty name, or None for Exit."""
```

```
    labels = [(name, f'{i + 1} - {name}') for i, name in enumerate(DIFFICULTIES)]
```

```
    labels.append((None, "Esc - Exit"))
```

```
btn_w, btn_h, gap = 260, 48, 16
```

```
total_h = len(labels) * btn_h + (len(labels) - 1) * gap
```

```
top = self.height // 2 - total_h // 2 + 30
```

```
buttons = []
```

```
for i, (action, label) in enumerate(labels):
```

```
    rect = pygame.Rect(0, 0, btn_w, btn_h)
```

```
    rect.centerx = self.width // 2
```

```

        rect.y = top + i * (btn_h + gap)

        buttons.append((action, label, rect))

    return buttons

python

def handle_event(self, event):

    # Movement is driven by the mouse position in handle_input;

    # events only matter once the round is over.

    if not self.game_over:

        return

    # Game over screen: any key or click opens the menu

    if not self.in_menu:

        if event.type in (pygame.KEYDOWN, pygame.MOUSEBUTTONDOWN):

            self.in_menu = True

            return

    # Menu: 1/2/3 or click to pick a difficulty, Esc or click to exit

    choice = "none"

    if event.type == pygame.KEYDOWN:

        names = list(DIFFICULTIES)

        if event.key == pygame.K_ESCAPE:

            choice = None

        elif pygame.K_1 <= event.key < pygame.K_1 + len(names):

            choice = names[event.key - pygame.K_1]

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:

        for action, _, rect in self._menu_buttons():

            if rect.collidepoint(event.pos):

                choice = action

                break

    if choice is None:

```

```

        pygame.event.post(pygame.event.Event(pygame.QUIT)) # main() quits cleanly
    elif choice != "none":
        self._start_round(choice)
python
def _render_menu(self, screen):
    title = self.big_font.render("Play Again?", True, WHITE)
    screen.blit(title, title.get_rect(center=(self.width // 2, self.height // 2 - 150)))

    mouse_pos = pygame.mouse.get_pos()
    for _, label, rect in self._menu_buttons():
        fill = (70, 70, 70) if rect.collidepoint(mouse_pos) else (40, 40, 40)
        pygame.draw.rect(screen, fill, rect, border_radius=8)
        pygame.draw.rect(screen, WHITE, rect, 2, border_radius=8)
        text = self.font.render(label, True, WHITE)
        screen.blit(text, text.get_rect(center=rect.center))
python
def render(self, screen):
    screen.fill(DARK)

    for wall in self.walls:
        pygame.draw.rect(screen, WALL_COLOR, wall.rect())

    pygame.draw.circle(screen, GOAL_COLOR, (self.goal_x, self.goal_y), self.goal_radius)
    pygame.draw.circle(screen, WHITE, (int(self.marble.x), int(self.marble.y)), self.marble.radius)

    elapsed = pygame.time.get_ticks() - self.start_ticks
    seconds_left = max(0, (self.time_limit_ms - elapsed) // 1000)
    timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
    screen.blit(timer_text, (10, 10))

    if self.game_over:

```

```

# Dim the maze behind the message
overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
overlay.fill((0, 0, 0, 200 if self.in_menu else 180))
screen.blit(overlay, (0, 0))

if self.in_menu:
    self._render_menu(screen)
    return

cx, cy = self.width // 2, self.height // 2

if self.result == "solved":
    title = self.big_font.render("You Win!", True, GOAL_COLOR)
    screen.blit(title, title.get_rect(center=(cx, cy - 50)))
    time_text = self.font.render(
        f"Finish time: {self.finish_time_ms / 1000:.1f}s", True, WHITE)
    screen.blit(time_text, time_text.get_rect(center=(cx, cy + 10)))
else:
    title = self.big_font.render("Time's Up!", True, WHITE)
    screen.blit(title, title.get_rect(center=(cx, cy - 30)))

prompt = self.font.render("Press any key or click to continue", True, WHITE)
screen.blit(prompt, prompt.get_rect(center=(cx, cy + 70)))

```

Notes:

- Restarting no longer calls `self.__init__`, since that would rebuild the fonts and maze and ignore the chosen difficulty. `_start_round` resets only the per-round state, and `__init__` now calls it with "Medium", so the first round plays exactly as before.
- Medium keeps your original numbers (0.6 / 0.02 / 45 s). Easy and Hard values are my starting guesses, so tune them in DIFFICULTIES.
- The game over prompt now says "continue" instead of "play again", since it leads to the menu.
- `update`, `handle_input` and `_resolve_wall_collisions` are untouched. The marble stays frozen during the menu because `game_over` remains True until a difficulty is chosen.

- Pressing a key on the game over screen only opens the menu. It doesn't also select an option, so pressing 1 there won't instantly start Easy.

Replay works. Relevant current code:

```
from game.py
```

```
def __init__(self, width, height):
```

```
    self.width = width
```

```
    self.height = height
```

```
    self.max_speed = 9
```

```
    self.walls = self._build_maze()
```

```
    self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22
```

```
    self.font = pygame.font.SysFont("Arial", 26)
```

```
    self.big_font = pygame.font.SysFont("Arial", 64, bold=True) # Game Over / menu title
```

```
    self._start_round("Medium")
```

```
def _resolve_wall_collisions(self):
```

```
    r = self.marble.radius
```

```
    for wall in self.walls:
```

```
        wall_rect = wall.rect()
```

```
        # Closest point on the wall rect to the marble's center
```

```
        closest_x = max(
```

```
            wall_rect.left, min(self.marble.x, wall_rect.right)
```

```
        )
```

```
        closest_y = max(
```

```
            wall_rect.top, min(self.marble.y, wall_rect.bottom)
```

```
        )
```

```
    dx = self.marble.x - closest_x
```

```

dy = self.marble.y - closest_y
dist_sq = dx * dx + dy * dy

if dist_sq >= r * r:
    continue # no collision (circle doesn't touch the wall)

if dist_sq > 0:
    # Center is outside the rect: normal points from closest point to center
    dist = math.sqrt(dist_sq)
    nx, ny = dx / dist, dy / dist
    penetration = r - dist
else:
    # Center is inside/on edge: push out through nearest side
    left = self.marble.x - wall_rect.left
    right = wall_rect.right - self.marble.x
    top = self.marble.y - wall_rect.top
    bottom = wall_rect.bottom - self.marble.y
    nearest = min(left, right, top, bottom)

    if nearest == left:
        nx, ny = -1.0, 0.0
    elif nearest == right:
        nx, ny = 1.0, 0.0
    elif nearest == top:
        nx, ny = 0.0, -1.0
    else:
        nx, ny = 0.0, 1.0
    penetration = nearest + r

# Push the marble out along the collision normal
self.marble.x += nx * penetration

```

```

self.marble.y += ny * penetration

# Reflect velocity component along normal if moving into wall
vn = self.marble.vx * nx + self.marble.vy * ny
if vn < 0:
    self.marble.vx -= (1 + 0.3) * vn * nx
    self.marble.vy -= (1 + 0.3) * vn * ny
def update(self):
    if self.game_over:
        return

    elapsed = pygame.time.get_ticks() - self.start_ticks
    if elapsed >= self.time_limit_ms:
        self.game_over = True
        self.result = "timeout"
        return

    self.marble.vx *= (1 - self.friction)
    self.marble.vy *= (1 - self.friction)

    speed = (self.marble.vx ** 2 + self.marble.vy ** 2) ** 0.5
    if speed > self.max_speed:
        scale = self.max_speed / speed
        self.marble.vx *= scale
        self.marble.vy *= scale

    self.marble.x += self.marble.vx
    self.marble.y += self.marble.vy

    self._resolve_wall_collisions()

```

```

gx = self.goal_x - self.marble.x
gy = self.goal_y - self.marble.y
if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
    self.game_over = True
    self.result = "solved"
    self.finish_time_ms = elapsed

```

Add sound effects: wall bounce, reaching the goal, and the timer running out. Generate short tones in code (no audio files), initialize pygame.mixer inside try/except so the game still runs without an audio device, throttle the bounce sound so it doesn't spam while rubbing a wall, and play the win/timeout sounds once. Then trace through the code and confirm the collision fix, game over screen, difficulty replay and sounds all work together; fix any conflicts. Give me only the changed functions, labeled with file names, and a short test checklist.

Coding synthesized envelopes and safe audio initialization fallbacks.

I traced all four features together and found one conflict, plus a few places where they interact safely. The conflict is in render: the HUD timer keeps counting down behind the Game Over overlay. After a win, elapsed keeps growing, so "Time: Xs" drifts to 0 even though you finished early. A small fix is included below.

game.py

Add near the top, with the other imports and constants (math is already imported):

python

```
from array import array
```

```
BOUNCE_COOLDOWN_MS = 120 # minimum gap between bounce sounds
```

```
BOUNCE_MIN_SPEED = 1.5 # impact speed below this is "rubbing", so stay silent
```

python

```
def __init__(self, width, height):
```

```
    self.width = width
```

```
    self.height = height
```

```
    self.max_speed = 9
```

```
    self.walls = self._build_maze()
```

```
    self.goal_x, self.goal_y, self.goal_radius = width - 60, height - 60, 22
```

```
self.font = pygame.font.SysFont("Arial", 26)

self.big_font = pygame.font.SysFont("Arial", 64, bold=True) # Game Over / menu title
```

```
self._init_sounds() # NEW

self._start_round("Medium")
```

python

```
def _tone_samples(self, rate, f0, f1, ms, volume):

    """Mono 16-bit samples for a sine sweep from f0 to f1 Hz, with a short fade in/out."""

    n = int(rate * ms / 1000)

    attack = max(1, int(rate * 0.005))

    samples = []

    phase = 0.0

    for i in range(n):

        t = i / n

        phase += 2 * math.pi * (f0 + (f1 - f0) * t) / rate

        env = min(1.0, i / attack) * (1 - t)

        samples.append(int(32767 * volume * env * math.sin(phase)))

    return samples
```

python

```
def _make_sound(self, samples, channels):

    data = array("h")

    for s in samples:

        for _ in range(channels):

            data.append(s)

    return pygame.mixer.Sound(buffer=data.tobytes())
```

python

```
def _init_sounds(self):

    """Build all sounds in code. If audio is unavailable, self.sounds stays empty."""

    self.sounds = {}

    self.last_bounce_ms = -BOUNCE_COOLDOWN_MS

    try:
```

```

if not pygame.mixer.get_init():
    pygame.mixer.init(22050, -16, 1, 512)
rate, fmt, channels = pygame.mixer.get_init()
if fmt != -16:
    raise RuntimeError("unsupported audio format")

tone = lambda f0, f1, ms, vol: self._tone_samples(rate, f0, f1, ms, vol)

bounce = tone(220, 150, 70, 0.5)
win = []
for freq in (523, 659, 784, 1047):    # C5 E5 G5 C6 arpeggio
    win += tone(freq, freq, 110, 0.5)
timeout = tone(440, 110, 600, 0.5)    # falling sweep

self.sounds = {
    "bounce": self._make_sound(bounce, channels),
    "win": self._make_sound(win, channels),
    "timeout": self._make_sound(timeout, channels),
}

except Exception:
    self.sounds = {} # no audio device: play silently

python
def _play(self, name, volume=1.0):
    sound = self.sounds.get(name)
    if sound:
        sound.set_volume(volume)
        sound.play()

python
def _play_bounce(self, impact_speed):
    """Throttled, volume scales with impact speed. Resting/rubbing contact is silent."""
    if impact_speed < BOUNCE_MIN_SPEED:

```

```

        return

    now = pygame.time.get_ticks()

    if now - self.last_bounce_ms < BOUNCE_COOLDOWN_MS:

        return

    self.last_bounce_ms = now

    self._play("bounce", 0.3 + 0.7 * min(1.0, impact_speed / self.max_speed))

python

def _resolve_wall_collisions(self):
    r = self.marble.radius

    for wall in self.walls:
        wall_rect = wall.rect()

        # Closest point on the wall rect to the marble's center
        closest_x = max(wall_rect.left, min(self.marble.x, wall_rect.right))
        closest_y = max(wall_rect.top, min(self.marble.y, wall_rect.bottom))

        dx = self.marble.x - closest_x
        dy = self.marble.y - closest_y
        dist_sq = dx * dx + dy * dy

        if dist_sq >= r * r:
            continue # no collision (circle doesn't touch the wall)

        if dist_sq > 0:
            # Center is outside the rect: normal points from closest point to center
            dist = math.sqrt(dist_sq)
            nx, ny = dx / dist, dy / dist
            penetration = r - dist
        else:
            # Center is inside/on edge: push out through nearest side
            left = self.marble.x - wall_rect.left

```

```
right = wall_rect.right - self.marble.x
top = self.marble.y - wall_rect.top
bottom = wall_rect.bottom - self.marble.y
nearest = min(left, right, top, bottom)
```

```
if nearest == left:
    nx, ny = -1.0, 0.0
elif nearest == right:
    nx, ny = 1.0, 0.0
elif nearest == top:
    nx, ny = 0.0, -1.0
else:
    nx, ny = 0.0, 1.0
penetration = nearest + r
```

```
# Push the marble out along the collision normal
```

```
self.marble.x += nx * penetration
self.marble.y += ny * penetration
```

```
# Reflect velocity component along normal if moving into wall
```

```
vn = self.marble.vx * nx + self.marble.vy * ny
```

```
if vn < 0:
    self.marble.vx -= (1 + 0.3) * vn * nx
    self.marble.vy -= (1 + 0.3) * vn * ny
    self._play_bounce(-vn) # NEW
```

```
python
```

```
def update(self):
```

```
    if self.game_over:
```

```
        return
```

```
    elapsed = pygame.time.get_ticks() - self.start_ticks
```

```

if elapsed >= self.time_limit_ms:
    self.game_over = True
    self.result = "timeout"
    self._play("timeout") # NEW (runs once: update returns early once game_over)
    return

```

```

self.marble.vx *= (1 - self.friction)
self.marble.vy *= (1 - self.friction)

```

```

speed = (self.marble.vx ** 2 + self.marble.vy ** 2) ** 0.5

```

```

if speed > self.max_speed:
    scale = self.max_speed / speed
    self.marble.vx *= scale
    self.marble.vy *= scale

```

```

self.marble.x += self.marble.vx
self.marble.y += self.marble.vy

```

```

self._resolve_wall_collisions()

```

```

gx = self.goal_x - self.marble.x
gy = self.goal_y - self.marble.y
if (gx ** 2 + gy ** 2) ** 0.5 <= self.goal_radius:
    self.game_over = True
    self.result = "solved"
    self.finish_time_ms = elapsed
    self._play("win") # NEW

```

In render, replace only the line `elapsed = pygame.time.get_ticks() - self.start_ticks` with:

```
python
```

```

if self.game_over:
    # Freeze the HUD timer once the round ends

```

```
elapsed = self.finish_time_ms if self.result == "solved" else self.time_limit_ms
```

else:

```
elapsed = pygame.time.get_ticks() - self.start_ticks
```

What the trace confirmed

- **Sounds and collision fix:** The sound call sits inside the existing $vn < 0$ branch, so the geometry and reflection math are unchanged. vn is a local scalar captured before the velocity edit, so $-vn$ is the true impact speed.
- **Bounce throttle:** Resting contact against a wall produces an impact of about one frame of tilt (max 0.9 on Easy), which is below `BOUNCE_MIN_SPEED`, and the 120 ms cooldown covers corner hits that touch two walls in one frame.
- **Win and timeout once only:** Both sounds fire in update at the moment `game_over` flips, and update returns early afterwards, so there's no replay during the overlay or menu. `_start_round` resets `game_over`, so each new round can play them again.
- **Replay:** `_init_sounds` runs once in `__init__`, not in `_start_round`, so restarting doesn't rebuild sounds or re-init the mixer.
- **Exit:** The posted `QUIT` also reaches `handle_event`, which ignores it (choice stays "none"), and `main()` then calls `pygame.quit()`, which shuts down the mixer.
- **No audio device:** Any failure in mixer init or tone generation leaves `self.sounds` empty, and `_play` becomes a no-op.

Test checklist

1. Run with audio: a hard hit on a wall gives a short thud, and a light bump is quieter.
2. Press the marble into a wall by holding the mouse toward it: no repeated sound while rubbing.
3. Slam into a wall corner: a single thud, no machine-gun effect.
4. Reach the goal: the arpeggio plays once, the "You Win!" screen shows, and the HUD timer stays frozen at your finish time.
5. Let the clock run out: the falling tone plays once, "Time's Up!" shows, and the HUD reads 0s.
6. Press a key, then pick Easy/Medium/Hard (keys and clicks): the marble resets to the start with fresh timer and settings. Win/timeout sounds play again on the next round.
7. Press Esc or click Exit in the menu: the window closes with no errors.
8. Close the window with the X during play, on the game over screen, and in the menu: all quit cleanly.
9. Simulate no audio (e.g. set `SDL_AUDIODRIVER=dummy`, or disable the output device before launch): the game runs silently with no crash.